

CRISTOFF STAN

Senior Go Software Engineer – Backend, Distributed Systems, Cloud

Location: Tallinn, Harjumaa, Estonia | Email: cristoffstan@gmail.com
LinkedIn: <https://www.linkedin.com/in/cristoff-s-692816413> | Phone Number: +372 58619311

PROFESSIONAL SUMMARY

Senior Software Engineer with 10+ years delivering **B2B SaaS, e-commerce, and enterprise operations platforms** using **Go 1.14–1.24, Gin, gRPC, PostgreSQL, Kafka, Kubernetes, and AWS/GCP**; specializes in high-throughput APIs, concurrent services, event-driven systems, legacy modernization, cloud-native delivery, and production reliability that improve latency, scalability, engineering velocity, and operating efficiency.

TECHNICAL SKILLS

- **Go Engineering:** Go 1.14–1.24, Gin, Echo, Chi, Fiber, Go modules, generics, goroutines, channels, contexts, interfaces, reflection, structured concurrency, worker pools, synchronization, memory management and dependency injection.
- **Backend Development:** REST APIs, gRPC, Protocol Buffers, GraphQL, WebSockets, background workers, schedulers, streaming services, API versioning, pagination, rate limiting, idempotency, request validation and third-party integrations.
- **Architecture and Design:** Microservices, modular monoliths, Domain-Driven Design, clean architecture, hexagonal architecture, CQRS, event-driven architecture, transactional outbox, saga workflows, multi-tenancy and dependency inversion.
- **Data and Persistence:** PostgreSQL, MySQL, MongoDB, DynamoDB, Redis, Elasticsearch/OpenSearch, GORM, sqlc, database migrations, query optimization, indexing, connection pooling and distributed caching.
- **Messaging and Streaming:** Apache Kafka, RabbitMQ, AWS SQS, SNS, Google Pub/Sub, NATS, dead-letter queues, message deduplication, retry policies, consumer groups, event replay and schema evolution.
- **Cloud Platforms:** AWS EKS, ECS, Lambda, API Gateway, RDS, DynamoDB, S3, SQS, SNS, EventBridge, IAM and CloudWatch; GCP Cloud Run, GKE, Cloud SQL, Pub/Sub, Cloud Storage and Cloud Monitoring.
- **DevOps and Infrastructure:** Docker, Kubernetes, Terraform, Helm, GitHub Actions, GitLab CI, Jenkins, Argo CD, Linux, Nginx, infrastructure as code, GitOps, blue-green deployments, canary releases and automated rollbacks.
- **Testing and Quality:** Go testing package, Testify, GoMock, Ginkgo, Gomega, Testcontainers, Pact, k6, benchmark testing, fuzz testing, race detector, staticcheck, golangci-lint, integration testing and contract testing.
- **Observability and Reliability:** OpenTelemetry, Prometheus, Grafana, Datadog, New Relic, Sentry, Elastic Stack, structured logging, distributed tracing, pprof, flame graphs, SLI/SLO monitoring and incident response.
- **Security:** OAuth 2.0, OpenID Connect, JWT, RBAC, mTLS, API keys, secrets management, input validation, encryption, least-privilege IAM, dependency scanning, threat modeling and audit logging.
- **Senior Engineering Practices:** Technical leadership, architecture reviews, mentoring, code reviews, capacity planning, performance profiling, production readiness, root-cause analysis, Agile delivery and stakeholder communication.

WORK EXPERIENCE

BEVY

Senior Software Engineer

(September 2021 – June 2026)

Remote, Romania

- Architected multi-tenant backend platforms using **Go 1.14–1.24, Gin, gRPC, PostgreSQL, Redis, Kafka, Kubernetes, and AWS**, defining reusable service templates and operational standards across several product teams.
- Modernized a tightly coupled Java and Node.js platform into domain-oriented **Go microservices**, extracting account, billing, reporting, document, and notification capabilities through staged migration and backward-compatible APIs.
- Reduced p95 API latency by **42%** by replacing CPU-intensive services with Go, eliminating redundant serialization, optimizing PostgreSQL queries, and caching frequently accessed reference data in Redis.
- Resolved production memory growth and request stalls caused by leaked goroutines, unbounded channels, missing context cancellation, and oversized buffers by introducing bounded workers and lifecycle-aware shutdown handling.
- Implemented event-driven workflows with **Kafka, transactional outbox records, idempotent consumers, retry policies, dead-letter topics, and replay controls**, preventing duplicated notifications and inconsistent downstream updates.
- Designed low-latency internal communication with **gRPC and Protocol Buffers**, reducing payload overhead while maintaining versioned contracts and compatibility across independently deployed services.
- Migrated selected Java, Python, and Node.js workloads into Go through shadow traffic, differential testing, progressive traffic shifting, and rollback controls that minimized production migration risk.
- Built secure REST and gRPC APIs with **OAuth 2.0, OIDC, JWT, tenant-aware RBAC, mTLS, audit logging, request signing, and rate limiting** for customer and administrative operations.
- Strengthened CI/CD with **GitHub Actions, Docker, Kubernetes, Terraform, golangci-lint, Testcontainers, security scanning, and canary deployment checks**, automating validation from commit through production.
- Mentored engineers on Go conventions, concurrent programming, API boundaries, database performance, observability, code reviews, and incident analysis, improving backend ownership across distributed teams.

Syscom Digital

Software Engineer

(October 2019 – August 2021)

Remote, Romania

- Developed commerce and operations services using **Go 1.10–1.16, Gin, PostgreSQL, Redis, Kafka, and AWS**, supporting checkout, payment processing, fulfillment, inventory, and customer-service workflows.
- Implemented payment, refund, order-tracking, and webhook processors with strongly typed domain models, provider adapters, signature verification, transactional writes, and retry-safe execution.
- Reduced recurring payment and webhook failures by **30%** through idempotency keys, correlation IDs, provider-specific error classification, dead-letter handling, and controlled event replay procedures.
- Investigated inventory mismatches caused by concurrent updates and delayed payment events, then introduced optimistic locking, compensating jobs, transactional status changes, and scheduled reconciliation reports.
- Resolved throughput degradation caused by lock contention and uncontrolled goroutine creation by introducing bounded worker pools, partitioned processing, shorter critical sections, and explicit backpressure.

- Created reusable adapters for payment, shipping, tax, and customer-data providers using **Go interfaces, typed DTOs, and contract tests**, isolating external schema changes from core business services.
- Improved PostgreSQL performance through execution-plan analysis, composite indexes, prepared statements, batched writes, connection-pool tuning, and server-side pagination for high-volume endpoints.
- Expanded delivery automation with **Jenkins, GitLab CI, Docker, Go tests, race detection, static analysis, schema validation, and staged deployments** across development, QA, and production environments.
- Collaborated with product owners, QA engineers, support teams, and external vendors to translate ambiguous commerce requirements into explicit domain rules, API contracts, and failure scenarios.

Soft Build

Software Developer

(June 2016 – July 2019)

Remote, Romania

- Built backend services and command-line utilities using **Go 1.4–1.10, PostgreSQL, MySQL, Redis, RabbitMQ, and Linux** for commerce, booking, reporting, and business-administration platforms.
- Improved batch-processing throughput by **38%** by moving pricing, import validation, and report transformations from interpreted scripts into concurrent Go workers with controlled memory usage.
- Developed REST APIs for inventory, appointment scheduling, pricing, authentication, reporting, and customer management using clear handler, service, repository, and domain boundaries.
- Resolved duplicate submissions, stale inventory values, partial database updates, and race conditions by adding idempotency records, transactions, synchronization guards, and recovery jobs.
- Implemented asynchronous workers for email delivery, report generation, data imports, and external synchronization using RabbitMQ, bounded queues, retry limits, and failed-message reporting.
- Added unit, integration, benchmark, and concurrency tests around parsers, pricing rules, database repositories, authentication middleware, and critical API behavior to reduce regression risk.
- Containerized Go services with **Docker**, standardized Linux runtime configuration, and added Jenkins pipelines for repeatable builds, database migrations, deployment verification, and rollback preparation.
- Worked with senior engineers on SQL execution plans, memory profiling, goroutine diagnostics, deployment automation, code reviews, and production incident resolution across data-intensive applications.

Decima Digital

Software Developer

(November 2015 – April 2016)

Remote, Estonia

- Delivered lightweight backend services and operational utilities using **Go 1.3, Python, PostgreSQL, MySQL, JavaScript, and Linux** for customer-facing websites, administration tools, and data-processing workflows.
- Built REST-style endpoints and command-line utilities for form processing, content retrieval, notification routing, file imports, scheduled reports, and basic account-management operations.
- Fixed production defects involving malformed CSV files, encoding errors, partial uploads, duplicate requests, missing database values, and unclear error responses through stronger validation and transactions.
- Improved slow reporting operations by reviewing generated queries, adding targeted indexes, batching database writes, and replacing repeated record loops with streaming transformations.
- Developed Go workers for CPU-intensive file parsing and data normalization, using explicit error handling, bounded memory usage, and atomic output replacement to protect incomplete results.
- Participated in Linux deployment preparation, smoke testing, log analysis, database backup verification, and rollback planning, helping reduce release risk across several client applications.

- Collaborated with designers, project managers, and senior engineers to translate business requirements into maintainable backend modules, stable data contracts, and realistic delivery estimates.

Borna Technology

Software Developer Intern

(September 2014 – August 2015)

Tallinn, Estonia

- Assisted senior engineers with **Go 1.1, Python, JavaScript, PostgreSQL, MySQL, Git, and Linux** tasks across internal utilities, client websites, and basic data-driven applications.
- Built small command-line tools, validation utilities, data-import helpers, and JSON endpoints while learning Go conventions, structured error handling, debugging, and version-control practices.
- Supported backend development for form handlers, database queries, file-processing tools, administration pages, and simple REST-style services under guidance from experienced engineers.
- Investigated invalid requests, Go runtime errors, SQL failures, file-processing defects, and inconsistent JSON responses by reproducing problems and documenting reliable diagnostic steps.
- Contributed to QA checks, deployment notes, unit tests, code reviews, and technical documentation, establishing a practical foundation in backend engineering and production support.

EDUCATION

Tallinn University

Bachelor's Degree in Computer Science,

September 2011 – May 2014

CERTIFICATIONS

- AWS Certified Solutions Architect – Associate
- AWS Certified Developer – Associate
- Certified Kubernetes Application Developer — CKAD
- MongoDB Certified Developer Associate
- Professional Scrum Developer I — PSD I

PROJECTS

High-Throughput Event Processing Platform

- Designed a distributed transaction-processing platform using **Go, Kafka, gRPC, PostgreSQL, Redis, Kubernetes, and AWS EKS** for payments, order updates, notifications, and operational reconciliation.
- Implemented partition-aware consumers, idempotency records, transactional outbox publishing, dead-letter topics, schema validation, replay controls, graceful shutdown, and backpressure for predictable traffic handling.
- Added distributed tracing, consumer-lag dashboards, benchmark suites, load tests, and automated canary checks that verified throughput, correctness, and latency before production promotion.